中山大學
SUN YAT-SEN UNIVERSITY

数据抽象和类 I

中山大学计算机学院

主讲人：万海

邮箱：wanhai@mail.sysu.edu.cn

目 录

CONTENTS

01 抽象数据类型

02 类的声明（初步）

03 成员函数及实现

04 对象创建与使用

05 应用：向量

抽象 - 看图联想单词

苹果 或 Apples。奇葩一点，会想到“乔布斯”…

为什么？这涉及人们认知世界的基本能力，**抽象**

抽象 - 事物认知

人类面对复杂的世界，是用区别与其他事物的**特征**去**识别事物和描述事物**。

例如：生物课上，我们使用**生物分类树**去认识生物，了解生物。这样我们就知道猫、老虎具具有相同的特征，它们属于猫科。猫、老虎、猫科就是**概念（Concepts）**

概念的特征包括**数据特征，行为特征：**

数据特征，用描述事物的外观、状态等及其值

行为特征，描述事物会做什么

抽象 - 抽象数据类型

- 抽象（Abstraction）？
  - 抽象是从众多的事物中抽取出**共同的**、**本质性的**特征，而舍弃其非本质的特征的过程。
  - 抽象的过程在于透过事物的现象获得它的本质，并用**概念**、原理、规律的形式**描述**它。
  - 抽象的意义在于通过事物的**本质**，识别或区分事物。
- 抽象数据类型（Abstraction Data Type/ADT）？
  - 现实世界事物概念的一种数字化描述，即计算机语言的表示。描述内容包括：
  - 属性（property） - 关于事物数据特征
  - 方法（method）- 关于事物的行为与操作特征

抽象数据类型（ADT）

在程序设计中，对于被抽象的数据的描述，称为抽象数据类型（Abstract Data Type, ADT）。

一种ADT应具有

1 声明部分（说明该该ADT是什么及如何使用）：说明部分描述数据值的特性和作用于这些数据之上的操作。ADT的用户仅须明白这些说明，而无须知晓其内部实现。

2 实现部分。

抽象数据类型 - 以日期声明为例

类型

Date

数据

内部数据表示。可能是 time_t 类型 或 struct tm 类型 或 自定义的数据

操作

```
set(year,month,day)
int getYear()
int setYear()
… …
int getWeekdays(date)
int diff_days(date1,date2)
Date add_diff(date,diff)
string toString(format)
```

面向对象语言日期对象和string对象使用类似。代码片段例如：

```
Data d1;
d1.set(2023,12,7);
cout << d1.getDay();
Data d2=d1.add_diff(d1,-5);
cout << d2.toString();
```

抽象数据类型 - 解析

- 把 Date 设计为一种数据类型。
- 内部包含年月日等数据以及在这些数据上可进行的操作。
- 用户就可以利用Date类型定义多个对象（变量）。
- 用户可调用每个对象公开的操作，但无法访问每个对象中被隐藏的内部数据。
- 用户也无需关心对象各操作的具体实现。
- 因此：Date 是一种封装的数据类型。达到了**信息隐藏**和**封装**的目的。

回顾：C语言当中的结构体

C **数组**允许定义可存储相同类型数据项的变量，**结构体** 是 C 编程中另一种用户自定义的可用的数据类型，它允许您存储不同类型的数据项。

结构用于表示一条记录，我们继续以上面的日期为例，我们可能会关心：

- Year
- Month
- Day

但C语言中的 struct 只能包含变量，导致数据与操作的函数分离！

```
typedef struct {
    int year;
    int month;
    int day ;
} Date;
void Set(Date*, int, int, int );
int getMonth(Date*);
int getDay(Date*);
int getYear(Date*);
void Print(Date*);
void Increment(Date*);
void Decrement(Date*);
```

先看第一个例子

日期对象

类型申明/定义：Date.hpp

成员函数实现及客户端：01_Date.cpp

类声明、实现、使用分离，多文件编译：

Date.cpp, 02_clientDATE.cpp

类成员函数内联实现：

DATE_inline.hpp,03_clientDATE_inline.cpp

类的声明 - C++ 语法

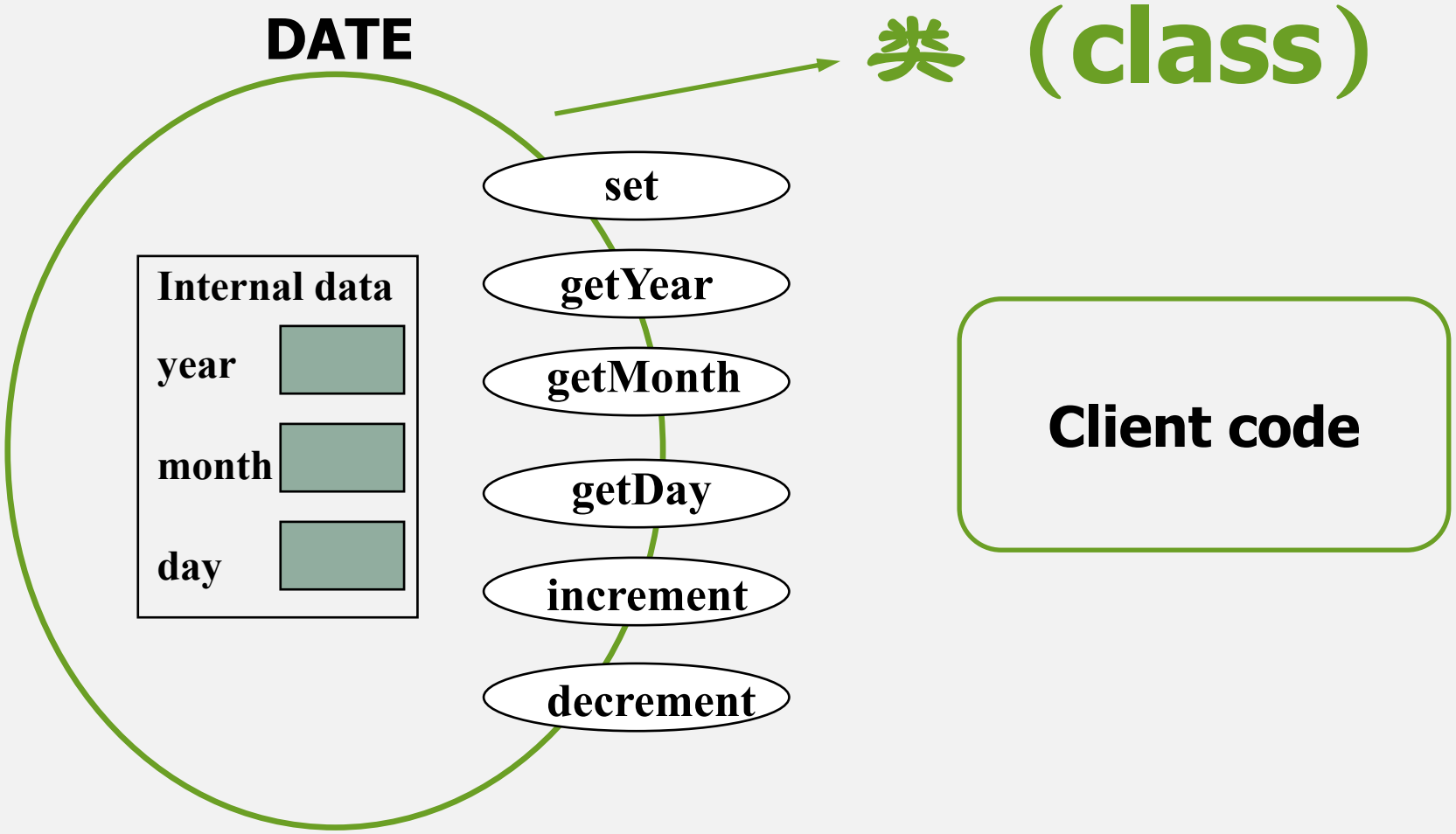
- 类关键词
  - class 或 struct 或 union 之一
- 访问说明符
  - public 公有，表示后续成员是对象使用者可访问
  - private 私有，表示后续成员仅对象内部可访问
  - protected 受保护
- 类成员
  - **数据成员**，按声明顺序存储，对齐规则同 C。一般设计中，数据成员需要隐藏保护。需要通过函数成员访问
  - **函数成员**，**公有成员**可以被外部使用者访问；**私有成员**仅内部访问；**const 函数成员**不能修改数据成员。

类声明比较复杂，这里仅是基本内容。详细见：类声明 - cppreference.com

类关键词 *类标识符* {
  访问说明符： （可选）
    数据成员声明序列;
    函数成员声明序列;
};

```
/* DATE.hpp*/
class DATE {
    public:
        void Set( int, int, int );
        int getMonth() const;
        int getDay() const;
        int getYear() const;
        void Print() const;
        void Increment();
        void Decrement();
    private:
        int month;
        int day;
        int year;
};
```

Date 类的实现（图示）


DATE
类（class）
set
getYear
getMonth
getDay
increment
decrement
Internal data
year
month
day
Client code

类的例子

类定义是以关键字 **class** 开头，后跟类的名称。类的主体是包含在一对花括号中。类定义后必须跟着一个分号或一个声明列表。例如，我们使用关键字 **class** 定义 Date 数据类型，如右所示。

```
class DATE {
    public:
        void Set( int, int, int );
        int getMonth() const;
        int getDay() const;
        int getYear() const;
        void Print() const;
        void Increment();
        void Decrement();
    private:
            int month;
            int day;
            int year;
};
```

类的例子

在{}中列出类的成员。类的成员包括：

- 数据成员：一般说来，数据成员是需要隐藏的对象；即外部的程序是不能直接访问这些数据的，应该通过函数成员来访问这些数据。所以一般情况下，数据成员通过关键字private声明为私有成员（private member）
- 函数成员：通过关键字public声明为公有成员（public member）。外部程序可以访问公有成员，但无法访问私有成员。

对于类的使用者（即用户代码，简称用户）而言，只需要获得 DATE.hpp，即可调用类对象的公有函数访问其内部的数据成员。使用者无法直接访问私有成员，也无需知晓公有函数的内部实现。

```
class DATE {
    public:
        void Set( int, int, int );
        int getMonth() const;
        int getDay() const;
        int getYear() const;
        void Print() const;
        void Increment();
        void Decrement();
    private:
            int month;
            int day;
            int year;
};
```

struct 与 class 关键字

C 语言：

- struct 是结构体类型关键字，结构体只能包含成员变量
- 结构体类型名称是 struct *identifer*

C++：

- 类关键包括 class，struct，union，除了可以包含成员变量，还可以包含成员函数
- 类型名称就是标识符，是 *xxx* 不是 struct *xxx*
- struct 定义的类成员默认是公有的
- class 定义的类成员默认是私有的。除此之外，class 和 struct 语法和语义没有区别

成员函数 - 声明

类的成员函数是指那些把定义和原型写在类定义内部的函数，就像类定义中的其他变量一样。类成员函数是类的一个成员，它可以操作类的任意对象，可以访问对象中的所有成员。

让我们看看之前定义的类 Date，现在我们要使用成员函数来访问类的成员，而不是直接访问这些类的成员。

```
class DATE {
    public:
        void Set( int, int, int );
        int getMonth() const;
        int getDay() const;
        int getYear() const;
        void Print() const;
        void Increment();
        void Decrement();
    private:
            int month, day, year;
};
```

成员函数 - 内联方式定义

成员函数可以定义在类**定义内部**，或者单独使用**作用域解析运算符 :: 来定义**。在类定义中定义的成员函数把函数声明为**内联**的，即便没有使用 inline 标识符。所以您可以按照如下方式定义 Set(int Y, int M, int D) 函数。

```
class DATE {
    public:
        void Set(int newYear,
                 int newMonth,
                 int newDay ) {
            month = newMonth;
            day = newDay;
            year = newYear;
        }
        ……
    private:
        ……
};
```

成员函数 - 外部定义

在类的外部使用**作用域解析运算符** :: 定义该函数，如右所示。

```
class DATE {
    public:
        void Set(int newYear,
                     int newMonth,
                     int newDay );
        ……
    private:
        ……
};
void DATE::Set(int newYear,
                     int newMonth,
                     int newDay ) {
    month = newMonth;
    day = newDay;
    year = newYear;
}
```

DATE类中成员函数

```
#include "DATE.hpp"
#include <iostream>
using namespace std;

int DaysInMonth( int, int );

void DATE::Set(int newYear,

int newMonth,

int newDay )
{  }
```

```
int DATE::getMonth() const
{  }
int DATE::getDay() const
{  }
int DATE::getYear() const
{  }
void DATE::Print() const
{  }
void DATE::Increment()
{  }
void DATE::Decrement()
{  }
int DaysInMonth(  int mo,  int yr  )
{  }
```

成员函数外部实现

```
//DATE.cpp  the implementation of each member
//function of DATE.

void DATE::Set( int newYear,
                int newMonth,
                int newDay ) {
    month = newMonth;
    day = newDay;
    year = newYear;
}
```

成员函数外部实现

```
//DATE.cpp  the implementation of each member function of
//DATE.
int DATE::getMonth() const
{
    return month;
}
int DATE::getDay() const
{
    return day;
}

int DATE::getYear() const
{
    return year;
}
```

成员函数外部实现

```
//DATE.cpp  the implementation of each
member function of //DATE.
void DATE::Print() const
{
    switch (month)
    {
        case 1 : cout << "January";
                 break;
        case 2 : cout << "February";
                 break;
                        :
        case 12 : cout << "December";
    }
    cout << ' ' << day << ", " << year <<
endl << endl;
}
```

成员函数外部实现

```
//DATE.cpp  the implementation of each member function
of //DATE.
void DATE::Increment()
{
    day++;
    if (day > DaysInMonth(month, year))
    {
        day = 1;
        month++;
        if (month > 12)
        {
            month = 1;
            year++;
        }
    }
}
```

成员函数外部实现

```
//DATE.cpp  the implementation of each member function of DATE.
void DATE::Decrement()
{  day--;
    if ( day == 0 )
    {
        if( month == 1 )
        {
            day = 31;
            month = 12;
            year--;
        }
        else
        {
            month--;
            day = DaysInMonth( month, year );
        }
    }
}
```

普通函数实现

```
//DATE.cpp  the implementation of the auxiliary function
//DaysInMonth.
int DaysInMonth( /* in */ int mo, /* in */ int yr  )
{
    switch (mo)
   {
     case 1: case 3: case 5: case 7: case 8: case 10: case 12:
              return 31;
     case 4: case 6: case 9: case 11:
             return 30;
     case 2:
          if ((yr % 4 == 0 && yr % 100 != 0) ||yr % 400 == 0)
              return 29;
          else
              return 28;
    }
}
```

成员函数

在DATE.cpp文件开头需要加入预处理命令

#include “DATE.hpp”

这是因为在DATE.cpp中要用到用户自定义的标识符DATE，而它的声明在DATE.hpp中。在DATE.hpp中，各函数原型是在{}中的。根据标识符的作用域规则，它们的作用范围仅在类声明中，而不包括DATE.cpp。因此在DATE.cpp中需要利用作用域解释运算符“::”来指明这里的函数是类DATE里的成员函数。

DATE.cpp中有时还包括DATE内部要使用到的函数，例如DaysInMonth。这种函数并非对外公开供用户使用，因此可以将其声明为类的私有成员。若在该函数中没有涉及该类的数据成员，则无需将它们声明为类的成员。

类与对象

- 类
  - 是**用户定义的数据类型**，它表示一个 ADT。在 C++中，它有属性（数据成员）和能操作数据成员的行为（函数成员）
  - 本质上是定义一个数据类型的蓝图，或者是一个模板。并没有实际数据
- 对象
  - 类定义的变量称为**对象（objects）**或者类的**实例（instances）**
  - 自动对象或动态申请的对象的**默认初始化值**为**不确定**。
- 客户端
  - 使用类的软件称为客户端代码。
  - 客户端代码使用对象的公有方法去处理对象的数据。

创建对象

结构体和类都可以看做一种由用户自己定义的复杂数据类型，在C语言中可以通过结构体名来定义变量，在 C++ 中可以通过类名来定义变量。不同的是，通过结构体定义出来的变量传统上叫变量，而通过类定义出来的变量有了新的名称，叫做对象（Object）。

在右边代码中，我们先通过 class 关键字定义了一个类 Date，然后又通过 Date 类创建了一个对象 date。

```
class DATE {
    public:
        void Set( int, int, int );
        int getMonth() const;
        int getDay() const;
        int getYear() const;
        void Print() const;
        void Increment();
        void Decrement();
    private:
            int month;
            int day;
            int year;
};
DATE date;
```

访问成员变量和函数

调用成员函数和成员变量是在对象上使用点运算符（.），这样它就能操作与该对象相关的数据，如下所示：

date.flag = 1;

date.set(2022,1,30);

```
class DATE {
    public:
        void Set(int newYear,
                     int newMonth,
                     int newDay ) {
            month = newMonth;
            day = newDay;
            year = newYear;
        }
        int flag;
        ……
    private:
        ……
}date;
```

访问成员变量和成员函数例子

```
//client.cpp
#include "DATE.h"
#include <iostream>
using namespace std;
int main()
{
    DATE date1, date2; //①
    int tmp;

    date1.Set( 1999, 10,
1 );
    date1.Print();
    date1.Increment();
    date1.Print();
                    :
}
```

```
    date2.Set( 1997, 7, 1 );
    date2.Print();
    date2.Decrement();
    date2.Print();

    tmp = date1.getYear();
    tmp++;
    date1.Set( tmp, 12, 20 );
    date1.Print();

    cout << date1.year;

//error?

    return 0;
}
```

练习：Vector3

Vector3 是 X 系统内定义的数据类型，部分描述如下：

- 描述
  - 用于表示 3D 向量和点。
  - 使用该结构传递 3D 位置和方向。 此外，它还包含用于执行常见向量操作的函数。
- 数据成员（公有）
  - x：向量的 X 分量。
  - y：向量的 Y 分量。
  - z：向量的 Z 分量。
- 公共函数
  - getMagnitude：返回该向量长度。
  - set：设置现有 Vector3 的 x、y 和 z 分量。
  - ToString：Returns a formatted string for this vector.

练习：Vector3

请完成以下任务：

- 请选择类描述符 class,struct,union 之一，声明Vector3。
- 给出定义文件 vector3.hpp
  - 浮点数使用 float 类型
- 给出成员函数实现 vector3.cpp
  - toString() 按 "%.2f %.2f %.2f" 格式返回关于 x, y, z 的字串
  - toString(string fm) 按指定格式返回该向量格式化字符串
- 编写客户端程序 point.cpp
  - 创建对象 p 点 (1,1,1)
  - 输出该对象 (1.00,1.00,1.00)

中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院

中山大学MOOC课程组